

Umoja Voices

Choir Management App — Design Plan

This document lays out the proposed design for a choir management web app: song library with per-voice-part media and structured lyrics, member accounts with admin/chorister roles, an invite-based signup flow, a travel logistics section for performance trips, and a public-facing external links page. It covers the data model, tech stack, page structure, and hosting recommendation. Once you've reviewed and adjusted this, we'll build a proof of concept.

1. Roles & Access

Two roles cover everything described: **Admin** and **Chorister**. There is no public/anonymous access to songs or member data — everything except the external links page requires login.

Capability	Chorister	Admin
View song list & song detail (players + lyrics)	Yes	Yes
Upload / edit / delete songs, voice parts, lyrics	No	Yes
View external links page	Yes	Yes
Manage external links	No	Yes
View logistics (deadlines, itinerary, practice schedule)	Yes	Yes
Manage logistics (create/edit trips, deadlines, itinerary, practice sessions)	No	Yes
Invite new members	No	Yes
View / edit member list, change roles, deactivate	No	Yes
Edit own profile (name, password)	Yes	Yes

Invite flow: an admin enters a new member's name and email on an *Invite Member* screen. The system creates a pending account and emails the invitee a secure, expiring link. They open it, set a password, and land in the chorister view already logged in. There is no public self-signup — accounts only exist via admin invite, which keeps membership limited to actual choir members.

Forgot password: a member who forgets their password doesn't need an admin's help to get back in. From the login screen they follow a *Forgot password?* link, enter their email, and — if it matches an active account — get a self-service reset email with a secure, expiring link to set a new password. This uses Supabase Auth's built-in password-reset flow (the same auth provider already handling invites and login), sent through Resend, so no custom token handling needs to be built for it.

2. Data Model

A relational schema (PostgreSQL) fits well since songs, parts, and lyrics sections are naturally ordered, related records. Proposed tables:

users

Column	Type	Notes
id	uuid	primary key
email	text, unique	login identifier
name	text	display name
role	enum: admin chorister	
status	enum: invited active disabled	
password_hash	text	set at invite acceptance
calendar_token	text, unique	random; secret ICS subscription identifier
invited_by	uuid, fk -> users.id	nullable
created_at / updated_at	timestamp	

invites

Column	Type	Notes
id	uuid	primary key
email	text	
token	text, unique	random, expiring
role	enum: admin chorister	role granted on acceptance
expires_at	timestamp	e.g. 7 days
accepted_at	timestamp	nullable

songs

Column	Type	Notes
id	uuid	primary key

Column	Type	Notes
title	text	
section_label	enum: S A T B SATB — Compulsory	which section the recording is assigned to
label_description	text, required	free text explaining the label — admin's own wording
created_by	uuid, fk -> users.id	
created_at / updated_at	timestamp	

song_parts

One row per voice part attached to a song (S, A, T, B, All — repeatable if a song has multiple takes).

Column	Type	Notes
id	uuid	primary key
song_id	uuid, fk -> songs.id	
part	enum: S A T B All	
label	text	e.g. "Soprano — full run"
media_url	text	external link or uploaded file URL
media_kind	enum: audio video youtube drive soundcloud direct_url	drives which embed player renders; direct_url is a plain-link fallback when nothing else matches
sort_order	int	display order within the song

lyric_sections

Lyrics are stored as an ordered list of labeled sections rather than one text blob, so the UI can render (and later re-order or re-label) verses/choruses independently.

Column	Type	Notes
id	uuid	primary key
song_id	uuid, fk -> songs.id	
section_type	enum: verse chorus bridge intro outro vamp custom	

Column	Type	Notes
section_label	text	e.g. "Verse 2" — auto-suggested, editable
content	text	the actual lyric lines
voice_tags	set of: S A T B SATB	which voice(s) sing this section; defaults to SATB
sort_order	int	

external_links

Column	Type	Notes
id	uuid	primary key
title	text	e.g. "Instagram", "Choir News Blog"
url	text	
category	enum: social news media other	groups the links page into sections
sort_order	int	

trips

A "trip" represents one performance travel engagement (e.g. a tour or festival abroad). Deadlines, itinerary, and practice sessions all hang off a trip so the app can show "what's next" and keep past trips archived.

Column	Type	Notes
id	uuid	primary key
title	text	e.g. "Portugal & Germany Tour 2026"
destination	text	e.g. "Portugal / Germany"
start_date / end_date	date	performance travel window, e.g. 2026-10-07 to 2026-10-18
status	enum: upcoming past	derived or manually set
created_by	uuid, fk -> users.id	

logistics_deadlines

Key dates members need to act on before a trip — visa applications, ticket purchases, etc.

Column	Type	Notes
id	uuid	primary key
trip_id	uuid, fk -> trips.id	
category	enum: visa tickets payment other	
label	text	e.g. "Visa application deadline"
due_date	date	
notes	text	nullable, extra instructions
sort_order	int	

itinerary_items

Column	Type	Notes
id	uuid	primary key
trip_id	uuid, fk -> trips.id	
date	date	
time	time	nullable
title	text	e.g. "Depart Nairobi", "Sound check", "Performance"
location	text	nullable
notes	text	nullable
sort_order	int	

practice_sessions

Rehearsal schedule leading up to a trip/performance. For the POC, each session is added as an individual dated entry (a weekly recurring rehearsal is added as several rows); true recurrence rules can be layered on later if useful.

Column	Type	Notes
id	uuid	primary key
trip_id	uuid, fk -> trips.id	nullable — supports general rehearsals not tied to a trip
date	date	

Column	Type	Notes
time	time	
location	text	
notes	text	nullable, e.g. "full choir", "tenors only"
sort_order	int	

3. Songs: Upload, Voice Parts & Embedded Players

When an admin creates or edits a song, the form has a title field, then a repeatable block per voice part (S / A / T / B / All) where they set a label and paste in the media link:

- **Pasted link** — YouTube, Google Drive share link, SoundCloud, or a direct .mp3/.mp4 URL. The app detects the link type automatically and stores the right *media_kind*.
- **Direct URL (fallback)** — if the pasted link doesn't match a recognized provider or file type, it's stored as a plain *direct_url* rather than rejected. The part still gets saved and shown, just as an "Open link" button instead of an inline embed — so any hosting the admin already uses works, even if it has no dedicated embed support.

Direct upload (recording a part straight into the app's own storage instead of pasting a link) is deliberately left out of the initial build — the POC works from pasted links only, confirmed in Section 11. Supabase Storage is still the right tool for it later; it just isn't wired up until it's needed.

On the chorister-facing song page, each part is shown as a labeled tab or row with an inline embedded player matched to its media kind — a YouTube iframe embed, a Google Drive preview iframe, a native HTML5 `<audio>/<video>` element for direct files, or a plain "Open link" button for the *direct_url* fallback — so playback happens without leaving the app.

Song section label

Separate from the per-part media above, each song is also tagged with one overall **section label** — S, A, T, B, or **SATB — Compulsory** — set by the admin when creating or editing the song. This is the "who is this song for / who must learn it" tag: a single-section label (e.g. "S") marks a piece as specific to that voice part (such as a descant), while **SATB — Compulsory** marks a song every member of the choir is required to learn.

Alongside the label, the admin writes a free-text **description** in their own words — there's no fixed system-generated caption, so the wording is entirely up to whoever is setting up the song (e.g. "Advanced soprano descant — optional" or "Required for all members, June concert opener"). The label renders as a colored badge on the song list and detail page, with the description shown alongside it, and the song list can be filtered by label so choristers can quickly find songs relevant to their own part or the compulsory set.

4. Structured Lyrics

Lyrics are entered and edited as a series of labeled sections rather than a single free-text box, so the structure (verse/chorus/bridge) is a real, editable property — not something inferred by guesswork every time the song is displayed.

Adding lyrics

- The admin adds sections one at a time: pick a type from a dropdown (Verse, Chorus, Bridge, Intro, Outro, Vamp, Custom), optionally override the auto-generated label ("Verse 1", "Verse 2", "Chorus"...), and type the lyric text into that section's box.

- Each section is also tagged with which voice(s) sing it — S, A, T, B, or SATB (the whole choir together) — using the same voice labels used elsewhere in the app. A section defaults to SATB unless the admin marks it as sung by a specific part (e.g. a soprano descant over the final chorus, or a bass solo line in a bridge). A section can carry more than one tag if two parts sing identical lyrics together apart from the rest of the choir.
- Sections can be reordered by drag-and-drop and deleted individually.
- **Quick-import** option: paste a full lyric sheet that already uses bracket tags (e.g. [Verse 1] / [Chorus]) and the app splits it into sections automatically (defaulting each to SATB), which the admin can then fine-tune, including adjusting voice tags.

Editing

Editing re-opens the same section list — an admin can retype the content of one section, change its type/label/voice tags, re-order sections, or add/remove sections, without touching the rest of the song.

Display to choristers

The song detail page renders each section under a small heading (e.g. "Verse 1", "Chorus"), in order, so singers can visually separate parts of the song while they listen to the matching voice-part recording alongside it. Each section also shows its voice tag(s) as a small badge (S / A / T / B / SATB).

Filtering lyrics by voice

Above the lyrics, the song page shows clickable filters for S, A, T, B, and SATB. Clicking one narrows the view to just the sections that voice sings — its own tagged sections plus any SATB (full-choir) sections, since those apply to everyone. This lets, say, an alto click "A" and see only the lyrics relevant to them instead of scanning the whole sheet. A "Full lyrics" option (the default view) clears the filter and shows every section.

5. External Links Page

A simple, admin-managed page grouping links by category — Social Media, News, Media/Press, Other — each shown with a title, icon, and outbound link. Visible to any logged-in member; editable only by admins from an /admin/links screen (add, reorder, remove).

6. Logistics & Travel

Since the choir travels to perform, members need one place to see the practical details of an upcoming trip — without being able to change any of it. Admins manage everything; choristers get a read-only view. A logistics page is built around the current (or next) *trip* and has three parts. The first real trip to seed the app with is the Portugal & Germany tour, 7–18 October 2026.

Key dates / deadlines

A simple checklist-style list of dates members must act on — e.g. "Visa application deadline — 2026-08-15", "Book flights by — 2026-08-01", "Final payment due — 2026-09-01" — each tagged by category (visa, tickets, payment, other) so they can be visually grouped or color-coded, with an optional notes field for instructions ("bring passport photos", "use the group booking code").

Itinerary

A day-by-day (and where relevant, time-by-time) schedule of the trip itself — travel legs, arrivals, rehearsals on site, sound checks, the performance(s), and departure — shown as an ordered timeline.

Practice schedule

The rehearsal calendar leading up to the trip — dates, times, and locations, with notes for section-specific rehearsals (e.g. "tenors only", "full choir dress rehearsal").

All three are admin-editable from /admin/logistics and view-only for choristers at /logistics — the same admin/chorister split used for songs and links.

Calendar sync & notifications

Practice sessions and itinerary items are date-based, so rather than expecting members to keep checking the /logistics page, the app pushes them into whatever calendar app each member already uses:

- **Personal ICS subscription feed** — each member gets a private, per-member calendar subscription URL (a random *calendar_token* embeds their identity, so it's not guessable, but no login/OAuth is required for the calendar app to read it). They add it once in Google Calendar, Outlook, or Apple Calendar's "subscribe by URL" option, and it then **auto-updates** whenever an admin adds, moves, or cancels a practice session or itinerary item — no re-adding needed.
- **Email notification with one-click add** — when an admin creates or changes a practice session (or itinerary item), affected members get an email (via the same transactional email service used for invites) with an attached/linked .ics file for that single event, so it can be added immediately even before they've set up the subscription feed.

This deliberately avoids OAuth-based direct writes into each member's Google/Outlook account: that would require every member to individually connect their calendar account, the app to store and refresh per-user access tokens, and (for Google specifically) app verification for calendar scopes — real ongoing security surface and maintenance for a problem the standard ICS format already solves, and ICS also works identically across Google, Outlook, and Apple Calendar rather than being tied to one provider.

7. Page / Route Map

Route	Access	Purpose
/login	public	Sign in; invite-acceptance flow lands here too
/accept-invite/[token]	public (valid token)	Set password, activate account
/forgot-password	public	Enter email to request a password reset link
/reset-password/[token]	public (valid token)	Set a new password

Route	Access	Purpose
/songs	chorister + admin	Browse / search / filter by section label (S, A, T, B, SATB — Compulsory)
/songs/[id]	chorister + admin	Voice-part players + structured lyrics
/links	chorister + admin	External links page
/logistics	chorister + admin	Current trip: key dates, itinerary, practice schedule (read-only)
/profile	chorister + admin	Edit own name, password
/admin	admin only	Dashboard: song count, member count, recent activity
/admin/songs, /new, /[id]/edit	admin only	Song CRUD incl. voice parts + lyrics editor
/admin/members, /invite	admin only	Member list, role changes, invite form
/admin/links	admin only	Manage external links
/admin/logistics, /[tripId]/edit	admin only	Manage trips, deadlines, itinerary, practice sessions
/api/calendar/[token].ics	token-authenticated	Per-member ICS subscription feed of practice sessions + itinerary

8. Recommended Tech Stack

Given the scale (a single choir, modest traffic, a handful of admins) the priority is low operational overhead over raw scalability. Recommendation:

Layer	Choice	Why
Framework	Next.js (App Router) + TypeScript	One codebase for pages + API routes; strong ecosystem; deploys natively to Vercel
Database	PostgreSQL via Supabase	Managed Postgres with built-in row-level security to enforce admin/chorister access at the DB layer
Auth	Supabase Auth	Built-in invite-by-email, password reset, session handling — avoids hand-rolling auth
File storage	Supabase Storage (deferred)	Not needed for the POC — pasted links only; add this only if/when direct upload is prioritized
ORM / data access	Prisma or the Supabase JS client	Type-safe queries against the Postgres schema above
Styling / UI	Tailwind CSS + shadcn/ui	Fast to build clean admin forms and a simple chorister-facing UI
Transactional email	Resend	Configured as Supabase Auth's custom SMTP provider, so invite links and password resets send through it (not Supabase's default mailer); also used for practice/itinerary change notifications

9. Hosting Recommendation

Option	Fit	Trade-off
Vercel (app) + Supabase (DB/Auth/Storage) — recommended	Excellent	Vercel's free/hobby tier comfortably covers a choir-sized app; Supabase free tier covers DB + auth + storage; near-zero ops; both scale up on the same platforms if the choir grows or wants multi-choir support later.

Option	Fit	Trade-off
Render (app + Postgres, one vendor)	Good	Single bill/vendor for app + database; free Postgres tier is more limited (expires after 90 days on the free plan) and cold-starts on the free web service tier are slower than Vercel's. Reasonable if you'd rather not depend on Supabase.
Netlify + Supabase	Good	Comparable to Vercel for Next.js hosting; Vercel has slightly better native Next.js support since they build the framework.
Self-hosted VPS (Hetzner/DigitalOcean)	Not recommended for POC	Full control, lowest recurring cost at scale, but you own patching, backups, and TLS — unnecessary overhead for a first version.

Recommendation: Vercel for the app, Supabase for database/auth/storage. At the confirmed scale of ~25 members and ~20 songs (Section 11), both platforms' free tiers cover this comfortably — no paid upgrade is needed to launch — and both deploy straight from git with no server management.

10. Proposed Proof-of-Concept Scope

To validate the core idea quickly, the first build would cover:

- Login + invite-acceptance flow (admin can invite one member by email) + self-service forgot-password flow.
- Admin: create/edit a song with title, section label (S/A/T/B/SATB — Compulsory) with a free-text description, one link per voice part (S/A/T/B/All) with label, and a structured lyrics editor (add/reorder/edit sections).
- Chorister: browse songs, open a song, play each voice part inline, read lyrics by section and filter them by voice (S/A/T/B/SATB).
- External links page with add/edit from the admin side.
- One trip's logistics: key dates/deadlines, itinerary, and practice schedule, admin-editable and read-only for choristers.
- Basic member list for admins (view + change role + deactivate).

Deliberately out of scope for the initial POC: bulk song import, mobile app, advanced search/tagging, recurring-rule practice scheduling, multi-trip history, and audit history. The ICS calendar feed and email notifications for practice sessions/itinerary (Section 6) are the first thing to add right after the core POC is validated, since they depend on wiring up the transactional email provider — everything else can wait longer.

11. Confirmed Decisions

- **Media for the POC:** pasted external links only (YouTube, Google Drive, SoundCloud, direct file URLs, or the `direct_url` fallback) — no file-upload UI or storage bucket wiring in the initial build; direct upload remains a documented fast-follow, not something to design out.

- **Branding:** pull the app's name styling, color palette, and logo cues from the existing @umojavoices Instagram profile (<https://www.instagram.com/umojavoices/>) rather than inventing new branding.
- **Scale:** ~25 members and ~20 songs to start. This is comfortably within the free tiers of both Vercel and Supabase, so no paid-tier upgrade is needed to launch the POC.
- **First real trip:** a Portugal/Germany performance trip, 7–18 October 2026, seeded as the first *trips* row (destination, key dates, itinerary, practice schedule) instead of placeholder data.

Prepared as a design plan for review. Next step after sign-off: scaffold the POC per Section 10 on the stack in Section 8, hosted per Section 9.